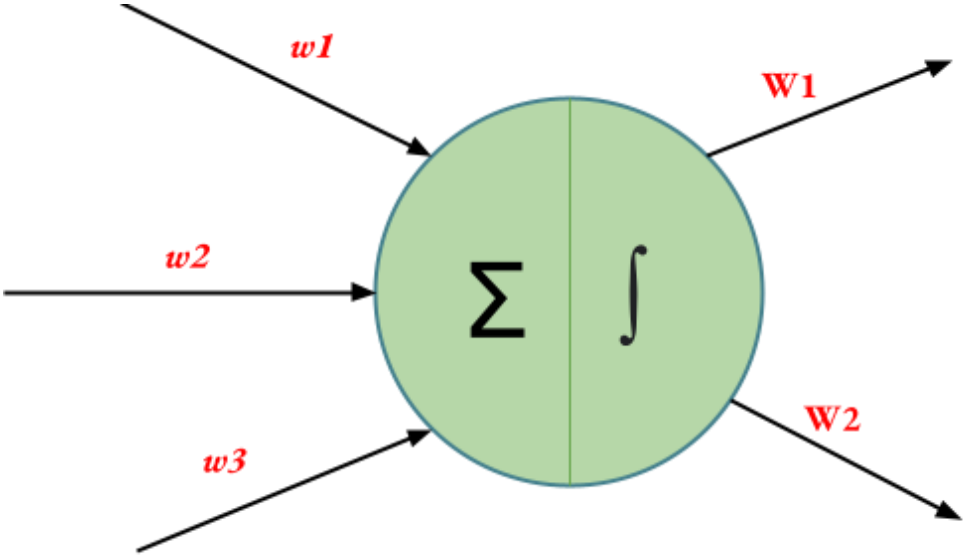
Weight Initialization Techniques for Deep Neural Networks

While building and training neural networks, it is crucial to initialize the weights appropriately to ensure a model with high accuracy. If the weights are not correctly initialized, it may give rise to the Vanishing Gradient problem or the Exploding Gradient problem. Hence, selecting an appropriate weight initialization strategy is critical when training DL models. In this article, we will learn some of the most common weight initialization techniques, along with their implementation in Python using *Keras* in *TensorFlow*.

As pre-requisites, the readers of this article are expected to have a basic knowledge of weights, biases and activation functions. In order to understand what this all, you are and what role they play in Deep Neural Networks - you are advised to read through the article Deep Neural Network With L – Layers

Terminology or Notations

Following notations must be kept in mind while understanding the Weight Initialization Techniques. These notations may vary at different publications. However, the ones used here are the most common, usually found in research papers.



Weight Initialization Techniques

1. Zero Initialization

As the name suggests, all the weights are assigned zero as the initial value is zero initialization. This kind of initialization is highly ineffective as neurons learn the same feature during each iteration. Rather, during any kind of constant initialization, the same issue happens to occur. Thus, constant initializations are not preferred.

2. Random Initialization

In an attempt to overcome the shortcomings of Zero or Constant Initialization, random initialization assigns random values except for zeros as weights to neuron paths. However, assigning values randomly to the weights, problems such as Overfitting, Vanishing Gradient Problem, Exploding Gradient Problem might occur.

3. Xavier

4. Normalized Xavier

Vanishing and Exploding Gradients Problems in Deep Learning

To train deep neural networks effectively, managing the Vanishing and Exploding Gradients Problems is important. These issues occur during backpropagation when gradients become too small or too large, making it difficult for the model to learn properly. Both problems directly affect the model's convergence and overall performance.

Vanishing Gradient Problem

Vanishing gradients occur when gradients become extremely small during backpropagation, causing early layers to learn very slowly or stop learning. During backpropagation the gradient of the loss L*L* with respect to a weight w*i*w*i* in layer i*i* is calculated using the chain rule:

When activation functions like Sigmoid or Tanh are used, their derivatives are less than 1. Repeated multiplication through layers causes the gradient to vanish as it moves backwards, making the lower layers unable to learn.

Exploding Gradient Problem

Exploding gradients occur when gradients grow too large during backpropagation, leading to unstable weight updates and divergence in loss. When derivatives or weights are greater than 1, their repeated multiplication across layers leads to exponential growth.

Why do the Gradients Vanish or Explode

- **Activation Functions:** Sigmoid or Tanh have small derivatives that shrink gradients.
- **Weight Initialization:** Too small or too large weights cause vanishing or exploding gradients.
- **Deep Networks:** Many layers multiply gradients repeatedly leading to instability.
- **Learning Rate:** High learning rate or unscaled inputs can make gradients explode.

1. Proper Weight Initialization

Choosing the right weight initialization keeps gradients balanced during backpropagation.

- **Xavier Initialization:** Keeps activation variance consistent across layers to stabilize gradients.
- **Kaiming Initialization:** Scales weights for ReLU to preserve signal strength and prevent gradient decay.

2. Use Non Saturating Activation Functions

Sigmoid and Tanh can shrink gradients. Using ReLU or its variants prevents vanishing gradients:

- **ReLU:** Basic rectified linear unit.
- **Leaky ReLU:** Allows small gradients for negative inputs.
- **ELU / SELU:** Helps maintain self normalizing properties.

3. Apply Batch Normalization

Normalizes layer inputs to have zero mean and unit variance, stabilizing gradients and accelerating convergence.

4. Gradient Clipping

Limits gradients to a maximum threshold to prevent them from exploding and destabilizing training.